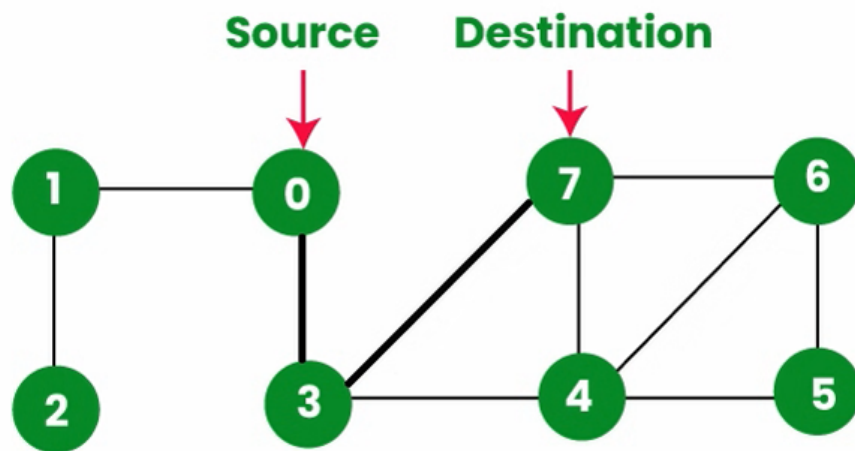


Shortest path in an unweighted graph

Last Updated : 11 Jul, 2025



Given an **unweighted, undirected** graph of **V** nodes and **E** edges, a source node **S**, and a destination node **D**, we need to find the shortest path from node S to node D in the graph.



Shortest Path in an Unweighted Graph



Input: $V = 8, E = 10, S = 0, D = 7$, $edges[][] = \{\{0, 1\}, \{1, 2\}, \{0, 3\}, \{3, 4\}, \{4, 7\}, \{3, 7\}, \{6, 7\}, \{4, 5\}, \{4, 6\}, \{5, 6\}\}$

Output: 0 3 7

Explanation: The shortest path is 0 -> 3 -> 7.

Input: $V = 8, E = 10, S = 2, D = 6$, $edges[][] = \{\{0, 1\}, \{1, 2\}, \{0, 3\}, \{3, 4\}, \{4, 7\}, \{3, 7\}, \{6, 7\}, \{4, 5\}, \{4, 6\}, \{5, 6\}\}$

Output: 2 1 0 3 4 6

Explanation: The shortest path is 2 -> 1 -> 0 -> 3 -> 4 -> 6.

Approach:

The idea is to use a modified version of [Breadth-First Search](#) in which we keep storing the parent of a given vertex while doing the breadth-first search. We first initialize an array $dist[0, 1, \dots, v-1]$ such that $dist[i]$ stores the distance of vertex i from the source vertex and array $par[0, 1, \dots, v-1]$ such

that `par[i]` represents the parent of the vertex `i` in the breadth-first search starting from the source.

Now we get the length of the path from source to any other vertex from array `dist[]`, and for printing the path from source to any vertex we can use array `par[]`.

Implementation:

C++ Java Python C# JavaScript

```
#include <bits/stdc++.h>
using namespace std;

// Modified bfs to store the parent of nodes along with the
// distance from source node
void bfs(vector<vector<int> >& graph, int S,
        vector<int>& par, vector<int>& dist)
{
    // queue to store the nodes in the order they are
    // visited
    queue<int> q;
    // Mark the distance of the source node as 0
    dist[S] = 0;
    // Push the source node to the queue
    q.push(S);

    // Iterate till the queue is not empty
    while (!q.empty()) {
        // Pop the node at the front of the queue
        int node = q.front();
        q.pop();

        // Explore all the neighbours of the current node
        for (int neighbour : graph[node]) {
            // Check if the neighbouring node is not visited
            if (dist[neighbour] == 1e9) {
                // Mark the current node as the parent of
                // the neighbouring node
            }
        }
    }
}
```

```

        par[neighbour] = node;
        // Mark the distance of the neighbouring
        // node as distance of the current node + 1
        dist[neighbour] = dist[node] + 1;
        // Insert the neighbouring node to the queue
        q.push(neighbour);
    }
}
}

// Function to print the shortest distance between source
// vertex and destination vertex
void printShortestDistance(vector<vector<int> >& graph,
                           int S, int D, int V)
{
    // par[] array stores the parent of nodes
    vector<int> par(V, -1);

    // dist[] array stores distance of nodes from S
    vector<int> dist(V, 1e9);

    // function call to find the distance of all nodes and
    // their parent nodes
    bfs(graph, S, par, dist);

    if (dist[D] == 1e9) {
        cout << "Source and Destination are not connected";
        return;
    }

    // vector path stores the shortest path
    vector<int> path;
    int currentNode = D;
    path.push_back(D);
    while (par[currentNode] != -1) {
        path.push_back(par[currentNode]);
        currentNode = par[currentNode];
    }

    // printing path from source to destination
    for (int i = path.size() - 1; i >= 0; i--)

```

```

        cout << path[i] << " ";
    }

    // Driver program to test above functions
    int main()
    {
        // no. of vertices
        int V = 8, E = 10;
        // Source and Destination vertex
        int S = 2, D = 6;
        // edge list
        vector<vector<int> > edges
            = { { 0, 1 }, { 1, 2 }, { 0, 3 }, { 3, 4 },
                { 4, 7 }, { 3, 7 }, { 6, 7 }, { 4, 5 },
                { 4, 6 }, { 5, 6 } };

        // vector to store the graph as adjacency list
        vector<vector<int> > graph(V);
        for (auto edge : edges) {
            graph[edge[0]].push_back(edge[1]);
            graph[edge[1]].push_back(edge[0]);
        }

        printShortestDistance(graph, S, D, V);
        return 0;
    }

```

Output

```
2 1 0 3 4 6
```

Time Complexity: $O(V + E)$, where **V** is the number of vertices and **E** is the number of edges.

Auxiliary Space: $O(V)$

 Comment

 A

ab_gu...

+ Follow

 109



Article Tags :

DSA

BFS

cpp-queue

Explore

DSA Fundamentals



Data Structures



Algorithms



Advanced



Interview Preparation



Practice Problem



Do Not Sell or Share My Personal Information